

AIL722 Assignment 1 Report

2024AIB2286 - Gaurav Meena

September 5, 2025

Contents

1	Question 1: Football Skills MDP	2
1.1	Part 1: Stationary Environment Analysis	2
1.1.1	Comparison for $\gamma = 0.95$	2
1.1.2	Analysis of Different Discount Factors	2
1.2	Part 1.3: Non-Stationary Environment Analysis	3
1.2.1	Algorithm Comparison	3
1.2.2	Performance Evaluation	3
1.3	Part 3: Potential Improvement in VI	3
1.4	Algorithm Comparison	4
1.5	Design Justification	4
2	Question 2: Dynamic Programming	5
2.1	Part 1: Online Knapsack Problem	5
2.1.1	Problem Formulation	5
2.2	Results and Analysis	5
2.2.1	Policy Iteration	5
2.2.2	Value Iteration	6
2.2.3	Analysis of Varying Episode Steps	8
2.3	Part 2: Portfolio Optimization	9
2.3.1	Problem Formulation	9
2.4	Results and Analysis	9
2.4.1	Value Iteration Results	9
2.4.2	Policy Iteration Results	10
2.4.3	Stochastic Environment Analysis	10

1 Question 1: Football Skills MDP

The assignment tested our understanding of making functions such as Policy Evaluation, Policy Improvement, Policy Iteration, and Value Iteration.

1.1 Part 1: Stationary Environment Analysis

This section analyzes the stationary football environment using Policy Iteration (PI) and Value Iteration (VI). The goal is to find an optimal policy for an agent to score a goal. Note: It is assumed that the assignment wants to explore the transition calls made by PI and VI and compare them hence, Transition cache has not been used. With transition cache the actual transition calls can be reduced

1.1.1 Comparison for $\gamma = 0.95$

For the primary discount factor of $\gamma = 0.95$, both algorithms were run to convergence. Table 1 summarizes the computational performance.

Table 1: Performance Comparison for $\gamma = 0.95$

Algorithm	Execution Time (s)	Iterations	Transition Calls
Policy Iteration	2.9070	26	1,002,400
Value Iteration	0.2838	30	86,800

Policy Comparison: The policies generated by both PI and VI were found to be identical. This is expected, as both algorithms are guaranteed to converge to the optimal policy for a given Markov Decision Process (MDP).

Performance Evaluation: The identical policies were then evaluated over 20 episodes, yielding the results in Table 2.

Table 2: Policy Evaluation for $\gamma = 0.95$

Algorithm	Mean Reward	Standard Deviation
Policy Iteration	47.00	21.79
Value Iteration	47.00	21.79

1.1.2 Analysis of Different Discount Factors

The analysis was repeated for $\gamma = 0.5$ and $\gamma = 0.3$. Table 3 summarizes the results. With effective caching, the number of unique transition function calls is the same for all algorithms, as they all must eventually learn the dynamics for the relevant parts of the state space. The primary difference remains in execution time and iteration counts.

This section details the implementation and comparison of Policy Iteration (PI) and Value Iteration (VI) on a stationary version of the environment.

Table 3: Comparison Across Different Discount Factors

γ	Algorithm	Time (s)	Iterations	Transition Calls
0.95	Policy Iteration	2.9070	26	1,002,400
	Value Iteration	0.2838	30	86,800
0.5	Policy Iteration	0.8595	26	280,000
	Value Iteration	0.2409	26	75,600
0.3	Policy Iteration	0.7233	26	226,800
	Value Iteration	0.1523	16	47,600

For $\gamma = 0.3$, the policies were not identical, which can happen due to tie-breaking behavior in states where multiple actions yield the same maximum value.

1.2 Part 1.3: Non-Stationary Environment Analysis

This section analyzes the environment in its non-stationary “degraded pitch” mode, where transition dynamics evolve over time.

1.2.1 Algorithm Comparison

A time-dependent Value Iteration algorithm was implemented to find a non-stationary policy $\pi(s, t)$. This was compared against a standard VI algorithm that assumes stationary dynamics. The standard VI was trained on the non-stationary environment, effectively learning a policy for the average dynamics.

Table 4: Non-Stationary Environment Algorithm Comparison

Algorithm	Execution Time (s)	Transition Calls
Time-Dependent VI	1.2190	112,000
Standard VI	4.0061	173,600

1.2.2 Performance Evaluation

The resulting policies were evaluated on the non-stationary environment. The time-dependent policy significantly outperformed the stationary one, demonstrating the benefit of adapting the policy to the changing dynamics of the environment.

Table 5: Policy Evaluation on Non-Stationary Environment

Policy Type	Mean Reward	Standard Deviation
Time-Dependent (Non-Stationary)	37.70	39.62
Standard (Stationary)	23.65	44.71

1.3 Part 3: Potential Improvement in VI

To improve upon the inefficiency of standard VI, a Prioritized Value Iteration algorithm was implemented.

1.4 Algorithm Comparison

This more efficient algorithm was compared against both standard VI and PI, using the updated, cached results for the standard algorithms. The results are shown in Table 6.

Table 6: Prioritized VI vs. Standard Algorithms ($\gamma = 0.95$)

Algorithm	Time (s)	Transition Calls	Updates/Iters
Prioritized VI	0.0869	27,727	401
Standard VI	0.2790	86,800	30
Policy Iteration	2.9328	1,002,400	26

An interesting result is that Prioritized VI now appears to make more transition calls than the standard algorithms. This is because the first step of this PVI implementation is to build a complete predecessor model of the environment, which requires iterating through all state-action pairs once (800 states * 7 actions = 5600 calls). The standard algorithms, with effective caching, only made 2,800 calls, indicating they converged without needing to explore the transitions for every single state-action pair.

Despite the higher initial cost of model-building, PVI is still significantly faster in execution time because it performs far fewer value updates (401) compared to the total updates in the full sweeps of standard VI (30 iterations * 800 states = 24,000).

1.5 Design Justification

The efficiency of Prioritized Value Iteration comes from the following design choices:

1. **Priority Queue:** A min-heap is used to store states prioritized by the magnitude of their Bellman error. This ensures that computation is focused on states that are furthest from their optimal value.
2. **Predecessor Model:** Before starting, the algorithm computes a reverse model of the environment. When a state's value is updated, this model allows the algorithm to efficiently find and queue all the states whose own values might now change.
3. **Focused Updates:** Unlike standard Value Iteration which sweeps through every state on each iteration, Prioritized VI only performs Bellman backups on states popped from the queue. This avoids wasting computation on states that have already converged or are not affected by recent updates.

2 Question 2: Dynamic Programming

2.1 Part 1: Online Knapsack Problem

2.1.1 Problem Formulation

The Online Knapsack problem requires an agent to maximize the total value of items in a knapsack without exceeding its weight capacity. The state is defined by (`current_weight`, `timestep`, `item_index`), and the agent can either accept or reject a randomly presented item at each step.

2.2 Results and Analysis

2.2.1 Policy Iteration

Policy Iteration was run for 5 iterations. The final knapsack values achieved across five different random seeds are presented in Table 7. The mean final value was 506.00 ± 104.64 .

Table 7: Policy Iteration Results for Online Knapsack

Seed	Final Knapsack Value
0	523.00
42	645.00
123	425.00
555	355.00
101	582.00

Figure 1 shows the value accumulation during evaluation, and Figure 2 visualizes the learned value function.

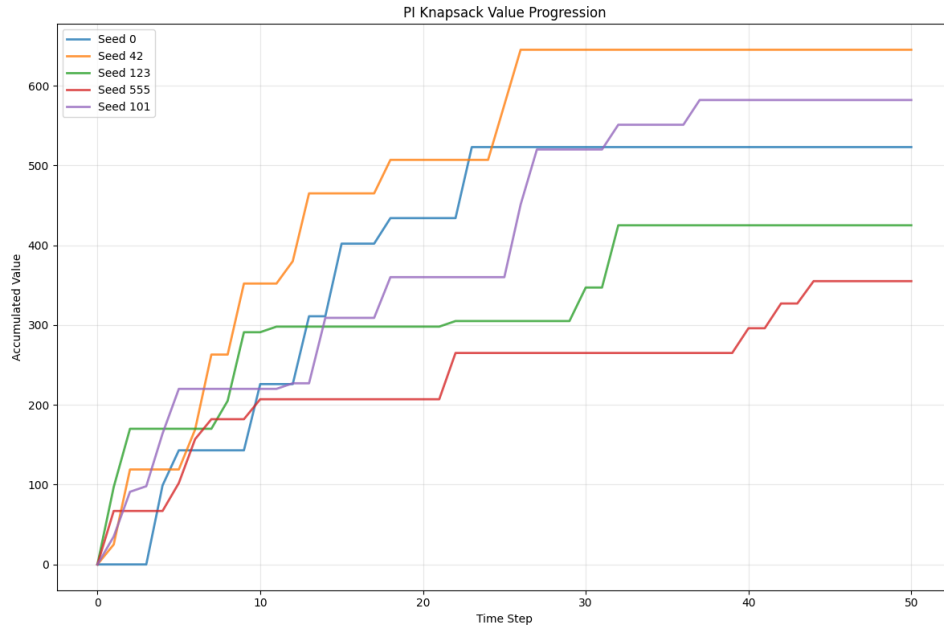


Figure 1: Policy Iteration: Knapsack value progression over 50 steps for 5 seeds.

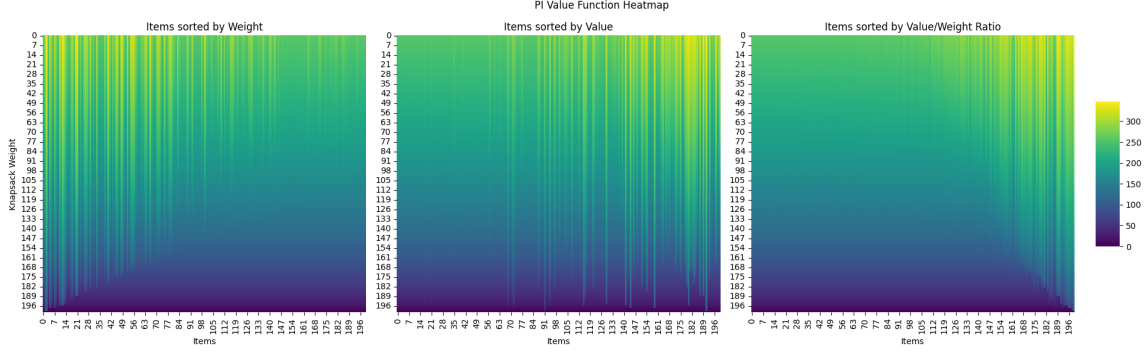


Figure 2: Policy Iteration: Heatmap of the value function, averaged over timesteps.

2.2.2 Value Iteration

Value Iteration converged after 51 iterations, taking approximately 730 seconds. The results across five seeds are shown in Table 8. The mean final value was 565.00 ± 73.64 , which is higher and more stable than Policy Iteration.

Table 8: Value Iteration Results for Online Knapsack

Seed	Final Knapsack Value
0	607.00
42	613.00
123	419.00
555	584.00
101	602.00

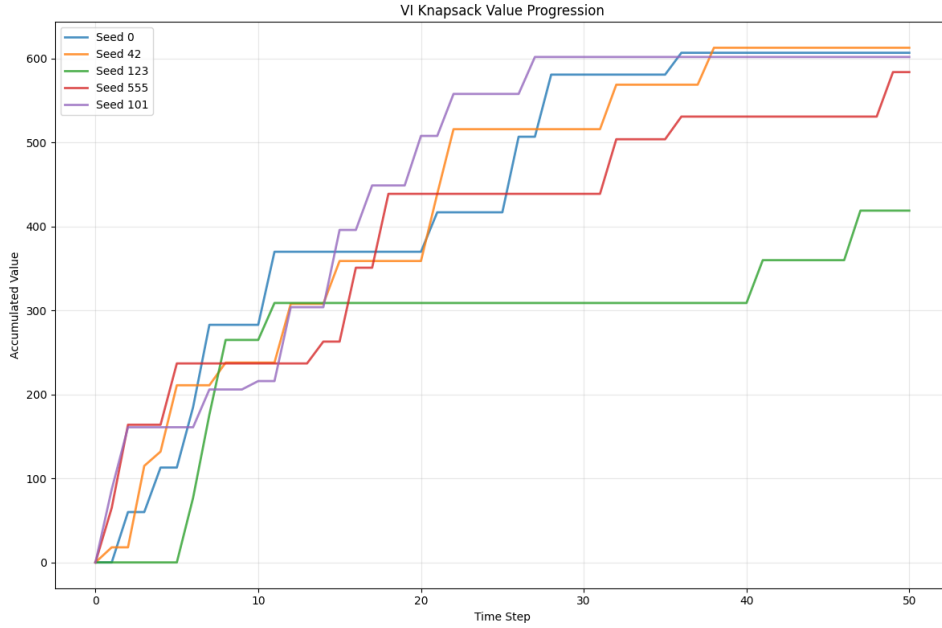


Figure 3: Value Iteration: Knapsack value progression over 50 steps for 5 seeds.

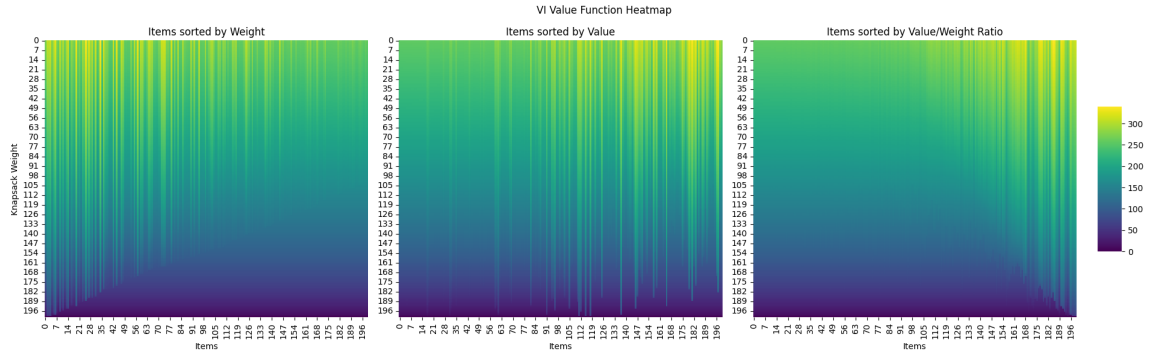
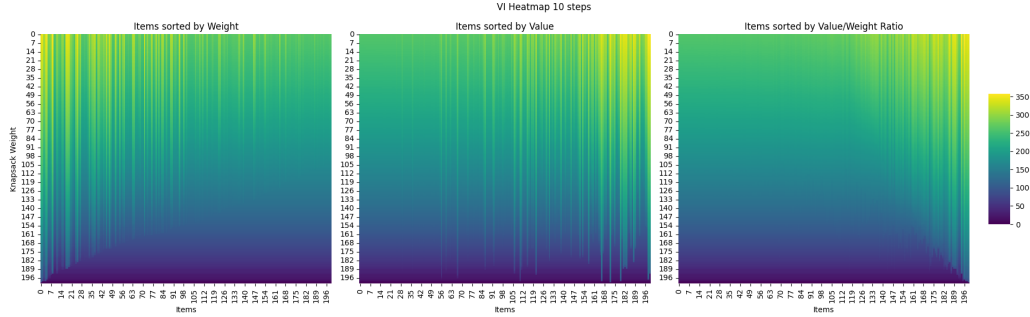


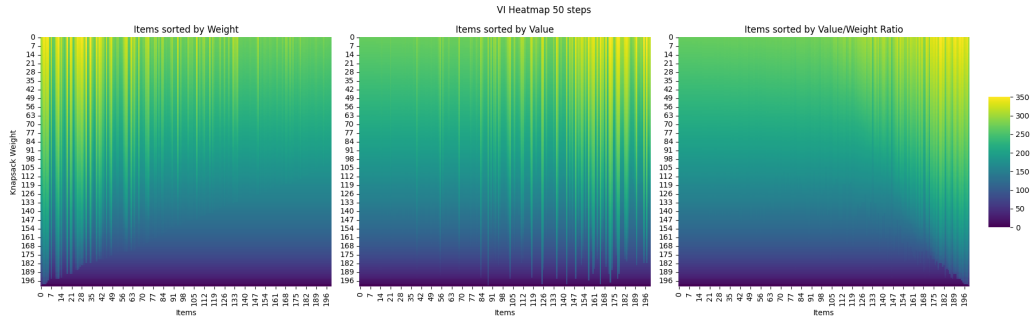
Figure 4: Value Iteration: Heatmap of the value function, averaged over timesteps.

2.2.3 Analysis of Varying Episode Steps

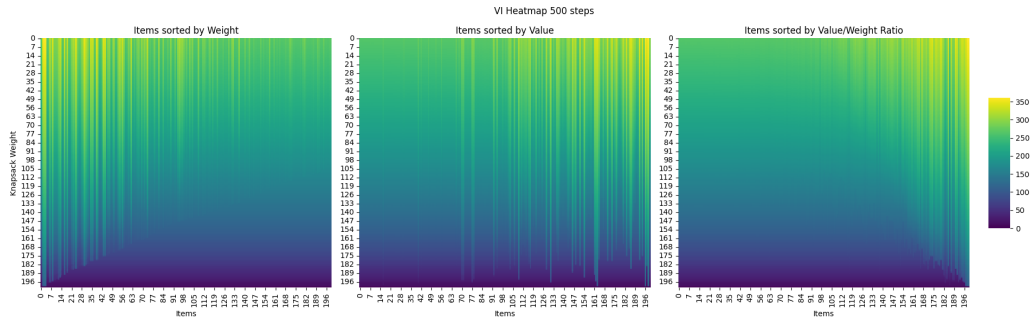
Value Iteration was run with episode lengths of 10, 50, and 500 steps. The resulting value function heatmaps are shown in Figure 5. As the time horizon increases, the value function shows a more pronounced gradient, indicating that with more steps remaining, the agent learns a more selective policy to save capacity for potentially high-value items that may appear later.



(a) 10 Steps per Episode



(b) 50 Steps per Episode



(c) 500 Steps per Episode

Figure 5: Value Iteration heatmaps for varying episode lengths.

2.3 Part 2: Portfolio Optimization

2.3.1 Problem Formulation

This problem involves maximizing the final wealth of a portfolio (cash + one asset) over a 10-period horizon. The state space (`timestep`, `cash`, `holdings`) is discretized to make the problem tractable for DP algorithms.

2.4 Results and Analysis

2.4.1 Value Iteration Results

Value Iteration was applied to three deterministic price sequences with discount factors (γ) of 0.999 and 1.0. The final wealth for each scenario is shown in Table 9. Figure 6 shows the detailed portfolio evolution for the first price sequence.

Table 9: Value Iteration: Final Portfolio Wealth

Price Sequence	$\gamma = 0.999$	$\gamma = 1.0$
Sequence 1	50.00	50.00
Sequence 2	20.00	20.00
Sequence 3	36.00	36.00

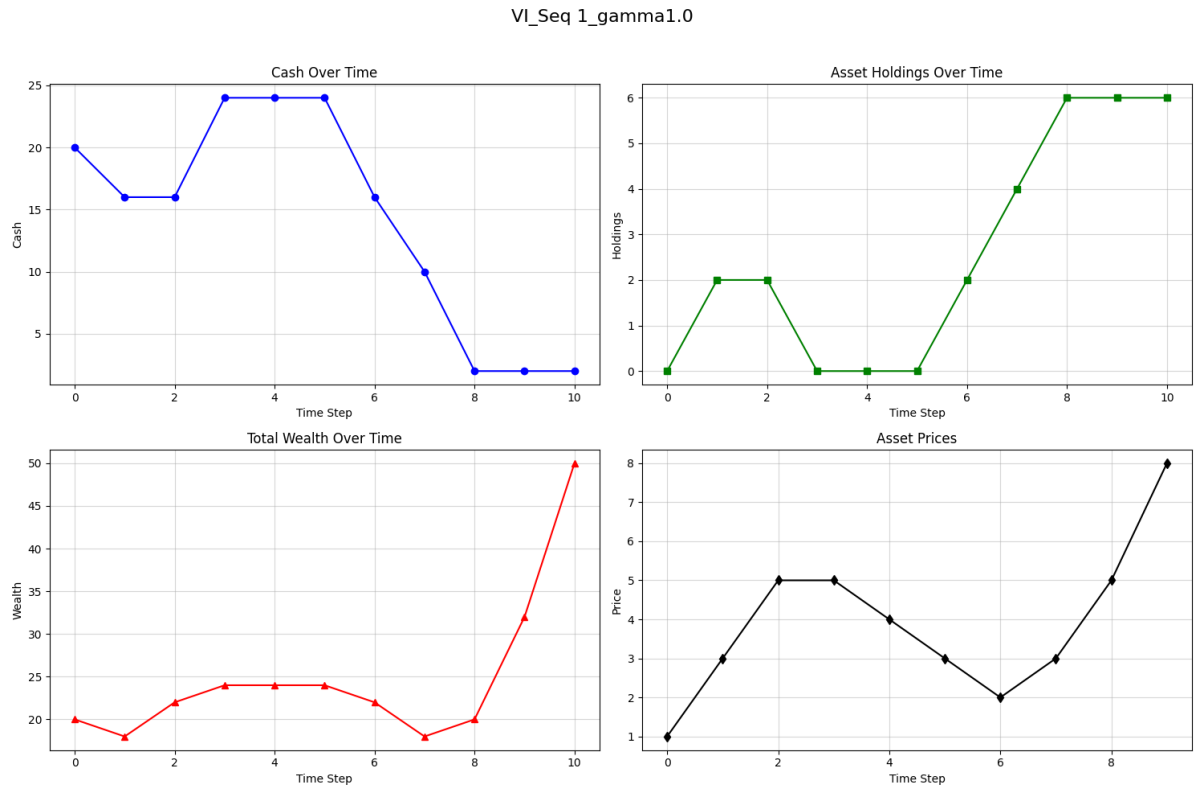


Figure 6: VI portfolio evolution for Price Sequence 1 ($\gamma=1.0$)

2.4.2 Policy Iteration Results

Policy Iteration yielded the same final wealth values as Value Iteration, as expected for a deterministic environment. The results are summarized in Table 10. Figure 7 shows the portfolio evolution for the first price sequence.

Table 10: Policy Iteration: Final Portfolio Wealth

Price Sequence	$\gamma = 0.999$	$\gamma = 1.0$
Sequence 1	50.00	50.00
Sequence 2	20.00	20.00
Sequence 3	36.00	36.00

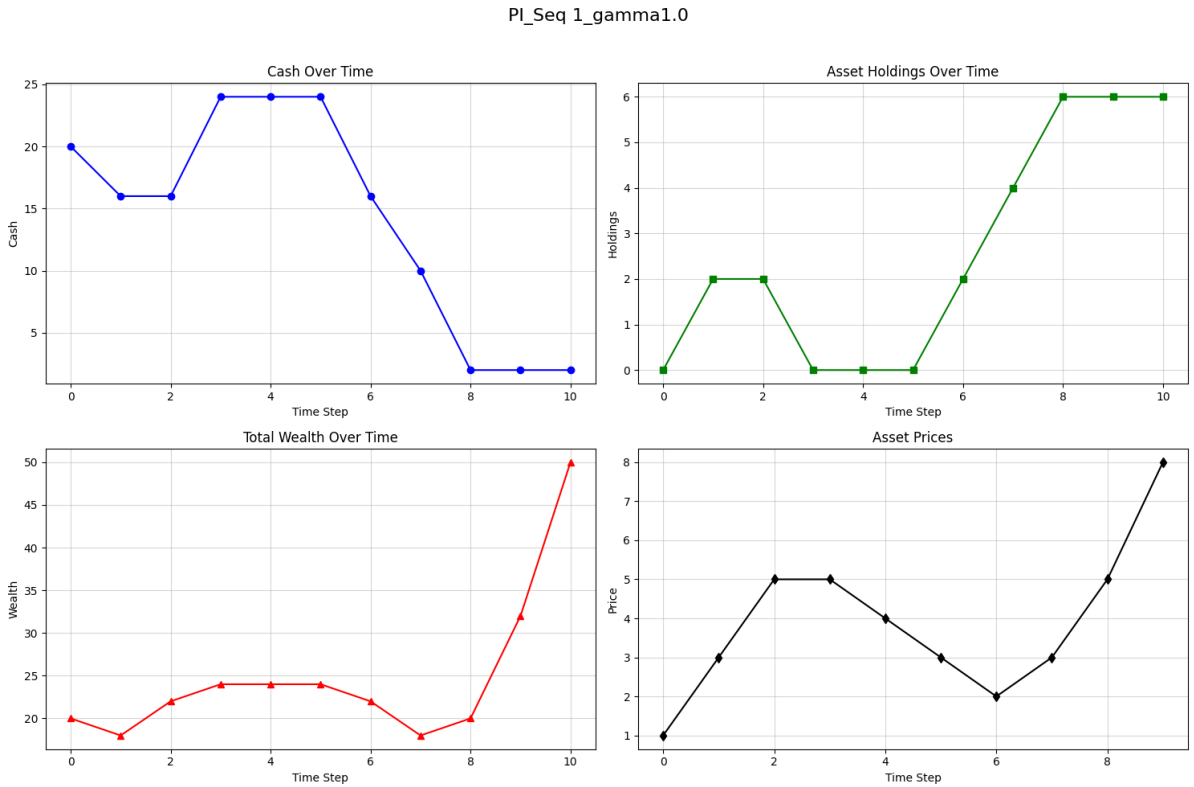


Figure 7: PI portfolio evolution for Price Sequence 1 ($\gamma=1.0$)

2.4.3 Stochastic Environment Analysis

For the final task, the environment was initialized with a price variance of 1.0. Policy Iteration was then run on the mean price path. The algorithm converged successfully in 3 iterations, as shown by the convergence plot in Figure 8. This demonstrates that the DP approach can find an optimal policy for the expected price trajectory in a stochastic setting.

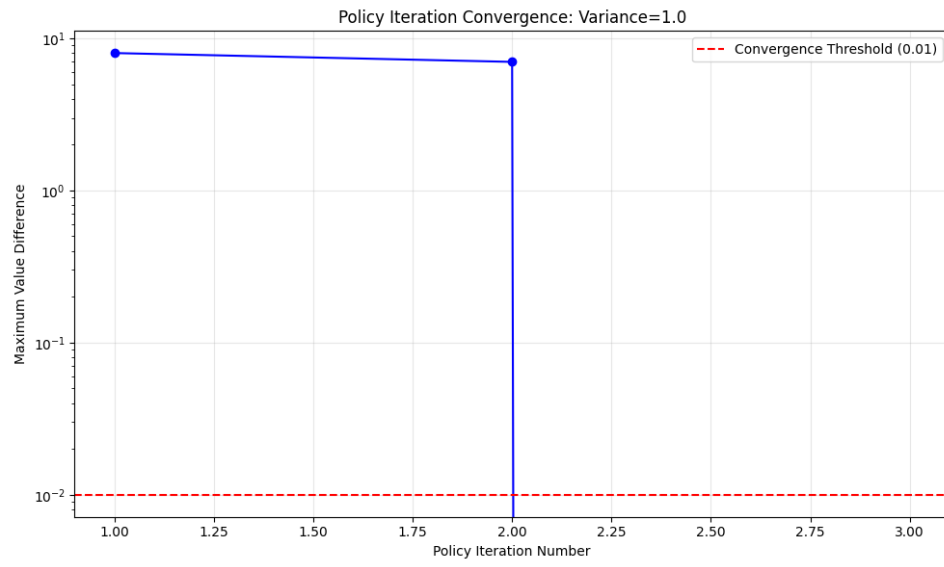


Figure 8: Policy Iteration convergence with price variance = 1.0.